
POLLITY

Building India's Last Mile of Digital Governance

MVP Plan & Architecture

Jan-Sunwai: Phase 0 to Beta Launch

Version: 1.0
Prepared by: Piyush Zaware, Co-Founder & CTO
Date: March 2026
Build Target: Q3–Q4 2026 Beta Launch
Beta Goal: 50 active users, NPS > 40
MVP Scope: Jan-Sunwai (grievance management only)

Build one thing well: **Jan-Sunwai**. A WhatsApp-first grievance intake and case management system for representative offices. Nothing else until 50 users are active and NPS > 40. Each additional agent (Vidhayak-Mitra, Vikas-Tracker, Nagrik-Setu) is roadmap — not MVP.

Contents

1	Technology Stack	2
2	MVP Scope: Jan-Sunwai v1	3
1	What Jan-Sunwai Does	3
2	What Jan-Sunwai Does NOT Do in v1	3
3	Grievance Categories (v1 NLP Taxonomy)	4
3	Build Phases	5
4	Milestones & Success Metrics	7
1	Key Performance Indicators (KPIs) During Beta	7
5	Repository Folder Structure	8
1	Top-Level Structure	8
2	Frontend: <code>apps/web/</code>	8
3	Backend: <code>apps/api/</code>	9
4	Infrastructure: <code>infra/</code>	10
5	Documentation: <code>docs/</code>	10
6	CI/CD: <code>.github/workflows/</code>	11
6	Core Data Model (PostgreSQL)	12
1	Grievance Status Lifecycle	12
7	Immediate Actions: What to Do This Week	13
1	Pre-Seed Budget Allocation (Technical)	13
8	Technical Risks & Mitigations	14

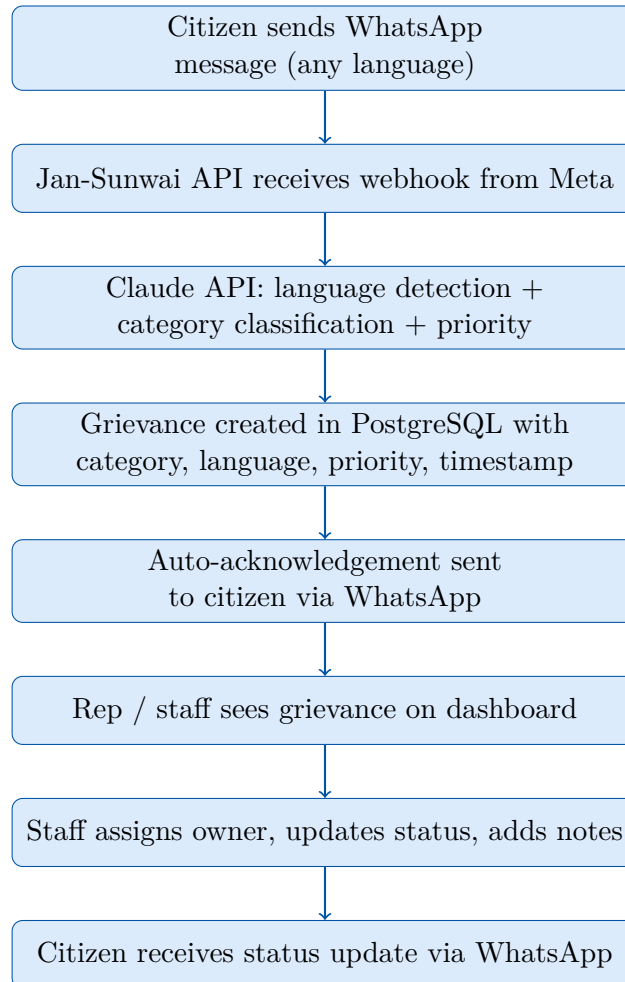
Technology Stack

Every choice below is made with three constraints in mind: (1) plays to the founding team's existing skills, (2) keeps costs near-zero during pre-seed, and (3) keeps data in India for DPDP Act compliance.

MVP Scope: Jan-Sunwai v1

1 What Jan-Sunwai Does

Jan-Sunwai is a WhatsApp-first grievance intake and case management system. The citizen never needs to open a website. The representative's office manages everything through a web dashboard.



2 What Jan-Sunwai Does NOT Do in v1

The following are explicitly out of scope for the MVP. Each item below is a future roadmap feature, not a v1 deliverable.

- CPGRAMS integration (requires 12–36 month government MOU negotiation)
- Vidhayak-Mitra: legislative research agent
- Vikas-Tracker: MPLADS/MLALADS fund management agent
- Nagrik-Setu: social media and constituency engagement agent
- Voice/IVR call intake
- SMS intake

- Fund tracking or budget analysis
- Social media monitoring or publishing
- Parliamentary procedure support

3 Grievance Categories (v1 NLP Taxonomy)

The Claude API classifier will map incoming grievances to 8 top-level categories, each with sub-categories to be refined during beta:

Build Phases

- Repository structure setup with full monorepo layout
- Docker + Docker Compose local development environment
- Supabase project: auth configuration + initial DB schema
- FastAPI skeleton: health check endpoint, CORS, logging
- Next.js dashboard skeleton: login page, protected routes, layout
- WhatsApp Business API sandbox: Meta developer account, test number
- GitHub Actions CI pipeline: lint, type-check, unit tests on every PR
- AWS Mumbai environment: VPC, EC2/ECS staging instance, RDS Postgres
- `.env.example` with all required environment variables documented

Exit criterion: Full stack runs locally with `docker-compose up`. Auth flow works end-to-end. WhatsApp sandbox receives test messages.

- WhatsApp webhook receiver with signature verification (HMAC-SHA256)
- Redis message queue: decoupled webhook ingestion from processing
- Claude API pipeline: language detection, category classification, priority scoring
- Multi-tenant grievance data model (one schema per rep office)
- Citizen auto-acknowledgement via WhatsApp (language-matched reply)
- Rep dashboard: grievance list with filters (status, category, date, priority)
- Grievance detail view: full conversation thread, metadata, action panel
- Status management: Open → In Progress → Resolved → Closed
- Citizen status notification: WhatsApp message on every status change
- Basic rep onboarding flow: office setup, WhatsApp number linking, staff invites

Exit criterion: A real WhatsApp message from a test citizen flows end-to-end through the system. Rep sees it on dashboard, updates status, citizen receives update.

- Bulk actions: assign, close, escalate multiple grievances simultaneously
- Dashboard analytics: volume trends, resolution time, category breakdown, SLA tracking
- Staff multi-user support: roles (Rep, PA, Constituency Manager, Viewer)
- Mobile-responsive dashboard (reps primarily on phones)
- Weekly email digest: summary of open grievances, resolution stats
- Data export: CSV download of all grievances with filters
- Search: full-text search across grievance content and citizen names
- Escalation flags: auto-flag grievances unresolved beyond configurable SLA

Exit criterion: 5 internal test users complete a full week of simulated grievance management without major usability issues.

- DPDP Act compliance: data retention policies, citizen consent flows, deletion requests
- Security baseline: OWASP Top 10 review, SQL injection, XSS, auth token audit
- Penetration testing: basic external pentest of API and dashboard
- ISO 27001 gap assessment (pre-work for future certification)
- Onboarding documentation for 50 beta users (rep + staff guides in English and Hindi)
- In-app NPS collection: PostHog survey after 7 days of active use
- Load testing: simulate 1,000 concurrent grievance submissions
- Bug bash: 5 internal testers covering all critical user journeys
- Staging → Production cutover on AWS Mumbai
- Beta user outreach: 50 representatives in Delhi and Maharashtra

Exit criterion: 50 beta users onboarded. Platform stable under load. NPS collection active. 5 documented case studies in progress.

Milestones & Success Metrics

1 Key Performance Indicators (KPIs) During Beta

Repository Folder Structure

The repository follows a **monorepo** architecture. All services — frontend, backend, infrastructure, and documentation — live in one repository. This maximises visibility for the full team, simplifies CI/CD, and makes contractor onboarding straightforward.

1 Top-Level Structure

```
pollity-mvp/
|
+-- apps/                                # All runnable applications
|   +-- web/                            # Next.js 14 frontend (rep dashboard)
|   +-- api/                            # FastAPI backend
|
+-- infra/                              # Infrastructure as code
|   +-- terraform/                      # AWS resource provisioning
|   +-- docker-compose.yml              # Local dev environment
|
+-- docs/                               # Technical documentation
+-- scripts/                            # Dev utility scripts
+-- .github/                            # CI/CD GitHub Actions workflows
+-- .env.example                        # All required env vars documented
+-- .gitignore
+-- docker-compose.yml                  # Root compose (full stack)
+-- README.md                           # Setup instructions for new developers
```

2 Frontend: apps/web/

```
apps/web/
+-- src/
|   +-- app/                            # Next.js App Router
|   |   +-- (auth)/                    # Route group: unauthenticated pages
|   |   |   +-- login/
|   |   |   +-- register/
|   |   |   +-- forgot-password/
|   |   +-- (dashboard)/                # Route group: authenticated pages
|   |   |   +-- grievances/             # Grievance list, detail, filters
|   |   |   +-- analytics/              # Volume charts, resolution KPIs
|   |   |   +-- settings/              # Office profile, team management
|   |   |   +-- layout.tsx              # Dashboard shell (sidebar, header)
|   |   +-- onboarding/                 # First-time setup wizard
|   |   +-- layout.tsx                  # Root layout (fonts, providers)
|   |   +-- page.tsx                    # Landing/redirect
|   +-- components/
|   |   +-- ui/                          # Base components (button, input,
|   |   badge)
|   |   +-- grievances/                  # Grievance-specific components
|   |   +-- analytics/                  # Chart components
```

```

| | +-- layout/                # Sidebar, header, breadcrumb
| +-- lib/
| | +-- api.ts                 # API client (typed fetch wrapper)
| | +-- auth.ts                # Supabase auth helpers
| | +-- utils.ts               # Shared utilities
| +-- types/
|   +-- grievance.ts           # Grievance TypeScript types
|   +-- office.ts              # Office/tenant types
|   +-- user.ts                # User/staff types
+-- public/                    # Static assets (logo, icons)
+-- package.json
+-- tsconfig.json
+-- next.config.js
+-- Dockerfile

```

3 Backend: apps/api/

```

apps/api/
+-- app/
|   +-- main.py                # FastAPI app entry point, router
|   registration
|   +-- core/
|   |   +-- config.py          # Pydantic settings (reads from .env)
|   |   +-- security.py        # JWT validation, Supabase auth
|   |   middleware
|   |   +-- database.py        # SQLAlchemy async engine, session
|   |   factory
|   |   +-- logging.py         # Structured logging setup
|   +-- modules/
|   |   +-- jan_sunwai/        # Grievance management module
|   |   |   +-- router.py      # FastAPI routes (/grievances/...)
|   |   |   +-- models.py      # SQLAlchemy ORM models
|   |   |   +-- schemas.py     # Pydantic request/response schemas
|   |   |   +-- service.py     # Business logic (CRUD + workflows)
|   |   |   +-- whatsapp.py    # Meta webhook handler + sender
|   |   +-- auth/
|   |   |   +-- router.py      # Auth routes (verify token, me)
|   |   |   +-- service.py     # Supabase auth integration
|   |   +-- offices/
|   |   |   +-- router.py      # Office/tenant CRUD routes
|   |   |   +-- models.py      # Office, staff, roles models
|   |   |   +-- service.py     # Multi-tenant isolation logic
|   |   +-- notifications/
|   |       +-- whatsapp_sender.py # Outbound WhatsApp message templates
|   |       +-- email_sender.py  # Weekly digest email
|   +-- workers/
|       +-- classifier.py       # Claude API NLP classification
|   pipeline

```

```

|      +-- queue.py           # Redis task queue (enqueue/dequeue)
|      +-- scheduler.py      # Cron jobs (weekly digest, SLA alerts)
+-- tests/
|   +-- unit/                # Unit tests per module
|   +-- integration/         # End-to-end API + DB tests
|   +-- fixtures/            # Test data factories
+-- alembic/                  # Database migrations
|   +-- versions/            # Migration files
|   +-- env.py
|   +-- alembic.ini
+-- requirements.txt          # Production dependencies
+-- requirements-dev.txt      # Dev/test dependencies
+-- Dockerfile

```

4 Infrastructure: infra/

```

infra/
+-- terraform/
|   +-- main.tf              # AWS provider, VPC, ECS, RDS
|   +-- variables.tf         # Configurable inputs (region, env)
|   +-- outputs.tf           # Exported values (ALB DNS, RDS
|       endpoint)
|   +-- modules/
|       +-- ecs/              # ECS Fargate task definitions
|       +-- rds/              # RDS Postgres (ap-south-1)
|       +-- redis/            # ElastiCache Redis
|       +-- alb/              # Application Load Balancer
+-- docker-compose.yml       # Local: Postgres + Redis + API + Web

```

5 Documentation: docs/

```

docs/
+-- architecture.md          # System design, data flow diagrams
+-- api-reference.md         # All API endpoints with examples
+-- onboarding.md            # New developer: clone to running in 15
    min
+-- data-model.md            # DB schema with field descriptions
+-- whatsapp-setup.md        # Meta Business API setup guide
+-- dpdp-compliance.md       # Data protection obligations
+-- grievance-taxonomy.md    # NLP category definitions

```

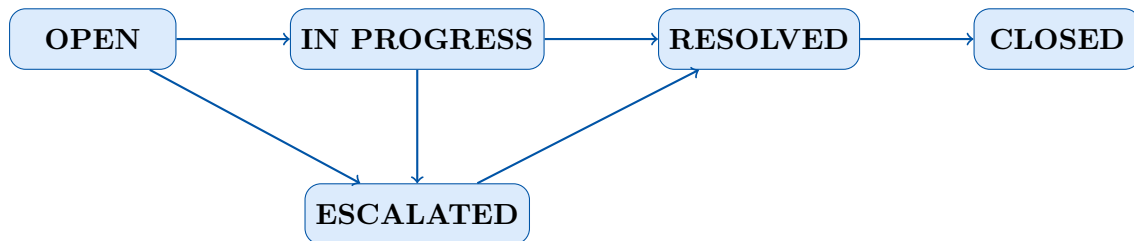
6 CI/CD: .github/workflows/

```
.github/  
+-- workflows/  
    +-- ci.yml                # On every PR: lint, typecheck, unit  
    tests                    tests  
    +-- deploy-staging.yml    # On merge to main: deploy to AWS  
    staging                  staging  
    +-- deploy-prod.yml       # On release tag: deploy to AWS  
    production
```

Core Data Model (PostgreSQL)

The four core tables for Jan-Sunwai v1. Multi-tenancy is enforced via `office_id` on every grievance row, combined with Supabase row-level security policies.

1 Grievance Status Lifecycle



Immediate Actions: What to Do This Week

These are the five tasks that unblock everything else. They must be done in this order.

1 Pre-Seed Budget Allocation (Technical)

Of the \$55K product development budget, the recommended technical spend:

Technical Risks & Mitigations

Table 1: Full Technology Stack

Layer	Choice			Reason
Backend	Python + FastAPI			Plays to CTO strengths; async-ready; native ML library support
Frontend	Next.js	14	(TypeScript)	App Router + server components; excellent for data dashboards; large contractor ecosystem
Database	PostgreSQL	via	Supabase	Multi-tenant row-level security; managed Postgres; free tier for development; built-in auth
Cache Queue	/	Redis		WhatsApp webhook message queue; rate limiting; fast session store
WhatsApp	Meta Business API			Direct webhook integration; no middleware cost; official channel for citizen communication
NLP / AI	Claude	API	(Anthropic)	Best-in-class multilingual understanding; handles Hinglish, Hindi, Marathi; context-aware classification
Auth	Supabase Auth			Row-level security built into Postgres; free tier; JWT-based; easy multi-tenant isolation
Hosting	AWS	Mumbai (ap-south-1)	(ap-south-1)	DPDP Act compliance — all citizen data must reside in India; lowest latency for Indian users
CI/CD	GitHub Actions			Free; native integration with repo; deploy on merge to main
Containers	Docker + Docker Compose			Dev/prod parity; enables contractor onboarding without environment setup friction
Monitoring	Sentry + PostHog			Sentry: error tracking + alerts; PostHog: product analytics, NPS, session recording

Table 2: Jan-Sunwai v1 Grievance Taxonomy

Category	Examples
Roads & Infrastructure	Pothole, streetlight, broken footpath, drainage
Water & Sanitation	Water supply, sewage, open defecation, cleaning
Electricity	Power outage, billing dispute, new connection
Health	Hospital access, medicine availability, ASHA worker
Education	School building, teacher absence, mid-day meal
Housing & Land	Slum demolition, property dispute, encroachment
Welfare Schemes	Ration card, pension, PM Awas, Jan Dhan
Law & Order	Harassment, noise, local crime, illegal construction

Table 3: MVP Milestone Tracker

Phase	Timeline	Key Deliverables	Success Metric
Phase 0	Weeks 1–3	Repo, Docker, auth, WhatsApp sandbox, CI/CD, AWS staging	Stack runs locally; CI green; WhatsApp sandbox active
Phase 1	Weeks 4–10	End-to-end WhatsApp grievance flow, dashboard, notifications	Full flow works; 0 critical bugs
Phase 2	Weeks 11–16	Bulk actions, analytics, multi-user, mobile, export	5 internal users complete 1 week simulation
Phase 3	Weeks 17–20	Security, DPDP, load test, onboarding docs, beta outreach	50 beta users onboarded; NPS > 40; 5 case studies
Seed Unlock	Month 9–12	Validated MVP, 50–100 active users	NPS > 40; DAU/MAU > 0.4; \$94K ARR path visible

Table 4: Beta KPI Dashboard

KPI	Definition	Target (Beta)
DAU / MAU	Daily active users / Monthly active users ratio	≥ 0.4
Grievance acknowledgement	Time from WhatsApp message to auto-reply	< 60 seconds
Grievance resolution time	Median time to Resolved status	< 5 days
NPS	Net Promoter Score from in-app survey	> 40
Rep retention	% of beta users still active at Week 8	$> 70\%$
Grievances processed	Total grievances logged across all beta offices	> 500
Time saved per rep/week	User self-report (onboarding vs. 8-week survey)	> 5 hrs/wk

Table 5: Core Tables

Table	Key Fields
offices	id, rep_name, whatsapp_number, tier, state
staff	id, office_id, name, email, role
grievances	id, office_id, citizen_phone, content_raw, content_translated, category, subcategory, p
messages	id, grievance_id, direction (inbound/outbound), content, wa_message_id, sent_at

Table 6: Week 1 Action Plan

#	Action	Why Can't Wait	It	Owner
1	Read the Jan-Sunwai system prompt PDF in full	Defines exact NLP classification logic and conversation flows before any code is written		Piyush
2	Read the Application Questions PDF	May contain accelerator/investor constraints that shape build decisions		Manav + Piyush
3	Scaffold the full repository structure (this document)	Unblocks any contractor or co-builder from starting immediately		Piyush
4	Apply for Meta WhatsApp Business API access	Meta verification takes 3–10 business days; delays the entire Phase 1 if not started now		Piyush
5	Create Supabase project + design grievance DB schema	Everything in Phase 1 depends on the data model being finalised first		Piyush

Table 7: Technical Budget Breakdown

Item	Budget	Notes
AWS Mumbai (EC2/ECS + RDS + ElastiCache + ALB)	\$12,000	12 months at approx. Rs.1,000/mo infra
Anthropic (Claude API) credits	\$6,000	Classification volume for 50 beta users + dev
WhatsApp Business API (Meta)	\$3,000	Message volume during beta
Supabase Pro tier	\$300	12 months, \$25/mo
Sentry + PostHog	\$600	12 months
Domain + SSL + misc SaaS tools	\$500	GitHub Teams, email, etc.
Frontend/backend contractor (6 weeks)	\$20,000	Senior full-stack contractor, Phase 1 sprint support
NLP contractor (4 weeks)	\$8,000	Hindi/Marathi classification fine-tuning
Security audit	\$4,600	Phase 3 pentest
Total	\$55,000	

Table 8: Technical Risk Register

Risk	Probabilit	Impact	Mitigation
Meta WhatsApp API approval delayed	Medium	High	Apply immediately; use Twilio WhatsApp sandbox as fallback for dev
Claude API classification accuracy low for informal Hinglish	Medium	High	Human-in-the-loop review queue for low-confidence classifications in v1
Supabase outage / data loss	Low	Critical	Daily automated backups to S3 Mumbai; multi-AZ RDS fallback plan
Multi-tenant data isolation breach	Low	Critical	Supabase RLS policies + integration tests that verify cross-tenant queries return empty
Contractor delivery delay	Medium	Medium	All work broken into 2-week sprint milestones with code review checkpoints
DPDP Act compliance gap	Medium	High	Engage legal counsel in Phase 3; conservative data retention defaults (90 days)
Redis queue overflow on high grievance volume	Low	Medium	Dead-letter queue + monitoring alert at 80% queue depth